

1. Can you explain the RAG architecture you implemented at MetLife?

At MetLife, I built a RAG pipeline mainly for claims and policy document search. We had a large number of documents, so I first extracted and cleaned the text, divided it into smaller chunks, generated embeddings, and stored those embeddings in Pinecone. When a user asked a question, I converted the question into an embedding, retrieved the most relevant chunks, and passed that context to the LLM through LangChain. This allowed the model to answer based on our actual documents instead of depending only on its training knowledge, and it helped reduce manual document review time by around 42%.

2. What is the difference between a traditional LLM application and RAG?

A traditional LLM answers mainly based on what it learned during training, so it may not know our latest or private company information. With RAG, I first retrieve relevant information from our own documents and then provide that information to the LLM as context. For example, if someone asks about a specific insurance policy, RAG retrieves the relevant policy sections first and then generates the answer. This improves accuracy and also reduces hallucinations.

3. How did you use LangGraph and CrewAI for multi-agent systems?

At MetLife, I used LangGraph and CrewAI to build multi-agent workflows around fraud detection. Instead of having one agent perform everything, I separated responsibilities. For example, one agent could collect claim information, another could analyze suspicious patterns, and another could help prepare the final risk assessment. LangGraph helped me control the workflow, state, and decision paths between agents. This approach helped us flag suspicious claims with around 91% precision.

4. How do you evaluate whether a RAG system is performing well?

I evaluate both retrieval and generation. For retrieval, I check whether the correct document chunks are appearing in the top results using metrics such as precision, recall, and retrieval relevance. For generation, I look at answer correctness, groundedness, hallucination rate, and whether the answer is supported by the retrieved context. I also monitor production metrics such as latency, error rate, token usage, and user feedback because a technically accurate system still needs to perform well for actual users.

5. Why did you use a vector database like Pinecone or ChromaDB?

I use a vector database because normal keyword search looks mainly for matching words, while vector search can understand semantic similarity. For example, a user might search for “car accident coverage,” while the document uses the phrase “automobile collision protection.” Semantic search can still recognize that those concepts are related. I have worked with Pinecone and ChromaDB for storing embeddings and retrieving relevant documents based on similarity.

6. How did you fine-tune and deploy LLMs at MetLife?

I worked with GPT-4o and Claude-based solutions for insurance use cases such as policy and underwriting summarization. My approach was to prepare domain-specific examples, evaluate the model against our expected outputs, and optimize prompts or model behavior based on accuracy and compliance requirements. For deployment, I used AWS services such as Bedrock and SageMaker and exposed capabilities through APIs so downstream applications could consume them. I also monitored quality, latency, and failures after deployment.

7. How did you monitor machine-learning models in production?

I used MLflow for experiment tracking, model versioning, and the model registry. At MetLife, we had around 15 production models, so monitoring was important. I tracked things like prediction distributions, model performance, latency, errors, and data drift. If the incoming data distribution changed significantly or model performance dropped below our threshold, we could investigate and decide whether retraining or another model update was required.

8. How did you reduce LLM inference latency?

I worked on several areas, including prompt optimization, reducing unnecessary context, improving retrieval, caching where appropriate, and model quantization for models where we controlled inference. For RAG specifically, I avoided passing too many document chunks to the LLM because that increases both latency and token cost. By improving retrieval and sending only useful context, we were able to reduce response times significantly while still maintaining answer quality.

9. Can you explain your YOLO project at Maruti Techlabs?

I worked on a warehouse inventory tracking project where we used YOLO-based object detection. The goal was to automatically identify inventory items from warehouse images instead of depending completely on manual tracking. I prepared and labeled image data, trained the YOLO model, evaluated detection performance, and tuned the model based on errors such as missed objects and false detections. We improved detection accuracy from approximately 78% to 94% during the client pilot.

10. How would you design a production-ready AI application?

I would separate it into different layers. First, I would have the data ingestion and preprocessing layer. Then I would have the AI or ML layer for retrieval, model inference, or agent execution. I would expose that through something like FastAPI and containerize the services using Docker. For scaling, I could deploy them through Kubernetes or cloud-managed services. I would also add authentication, logging, monitoring, model versioning, CI/CD, and fallback mechanisms

because production AI is not only about model accuracy; reliability and observability are equally important.

## Behavioral Interview Questions

1. Tell me about yourself.

I'm an AI/ML Engineer with more than four years of experience working across machine learning, generative AI, NLP, computer vision, and MLOps. Currently at MetLife, I work mainly on RAG systems, LLM applications, multi-agent workflows, and production model monitoring for insurance use cases. Before that, at Maruti Techlabs, I worked on conversational AI, computer vision, NLP, recommendation systems, and large-scale data pipelines. What I enjoy most is taking an AI idea from experimentation all the way to a production solution that solves an actual business problem.

2. Tell me about a challenging project you worked on.

One challenging project at MetLife was our claims RAG system because we were dealing with a large number of insurance documents and users expected very accurate answers. Initially, retrieval quality wasn't consistent because similar policies contained very similar language. I worked on improving preprocessing, chunking, metadata filtering, embeddings, and retrieval logic. I also added evaluation to understand where incorrect answers were coming from. After those improvements, we achieved better retrieval quality and reduced manual claims-document review time by around 42%.

3. Tell me about a time you worked with non-technical stakeholders.

At MetLife, I regularly worked with actuarial, underwriting, claims, and compliance teams. I avoided explaining everything in terms of embeddings, models, or vector databases. Instead, I focused on what the system was doing for them. For example, I would explain that the AI searches the approved policy documents first and uses that information to generate an answer. I would then show them examples and collect their feedback. That helped us translate business requirements into technical requirements.

4. Tell me about a time something didn't work as expected.

During one of my AI projects, the initial model performed well on our test dataset but its performance wasn't as consistent with real user inputs. I investigated the failures and found differences between our test data and actual production queries. I expanded the evaluation dataset with more realistic examples, improved preprocessing, and adjusted the model pipeline. It taught me that offline metrics alone aren't enough and that production data and user behavior need to be included in model evaluation.

5. How do you handle multiple priorities?

I normally prioritize based on business impact, production risk, dependency, and deadline. For example, if I have a new model feature and at the same time there is a production model showing performance degradation, I will address the production issue first because it is already affecting users. I communicate the impact to my team, break the work into smaller tasks, and use Jira to keep the priorities visible so everyone understands what I'm working on and why.

6. Tell me about a disagreement with a team member.

I try to make technical disagreements data-driven. For example, if one engineer prefers one embedding model and I prefer another, I don't want the decision to depend on personal preference. I would define an evaluation dataset and compare retrieval accuracy, latency, cost, and scalability. Then we can select the solution based on measurable results. I've found that this approach makes technical discussions much more productive.

7. Tell me about a time you improved an existing process.

At MetLife, model performance was previously being reviewed through more periodic checks, which meant degradation could sometimes take longer to identify. I helped establish MLflow-based model tracking and monitoring dashboards for around 15 production models. We added monitoring around model behavior and drift so that problems could be identified much earlier. That reduced the time to detect certain performance degradation from weekly reviews to within roughly 24 hours.

8. How do you deal with tight deadlines?

I first identify what is absolutely necessary for the initial delivery and separate that from improvements that can come later. I break the project into smaller deliverables and focus on getting an end-to-end working solution first. At the same time, I don't compromise on critical areas like security, testing, or data quality. I also communicate risks early instead of waiting until the deadline, because that gives the team time to adjust priorities if necessary.

9. Tell me about a project you're proud of.

One project I'm proud of is the multi-agent fraud detection workflow I worked on at MetLife. We used LangGraph and CrewAI to coordinate different AI tasks involved in analyzing suspicious claims. The interesting part was not simply using an LLM, but designing how different agents shared information and made decisions while keeping the workflow controlled. The solution achieved around 91% precision in flagging suspicious claims and was estimated to help prevent about \$1.2 million annually in losses.

10. Why are you interested in this AI/ML role?

I'm interested because the role matches the direction I've been building my experience in: production AI systems rather than only model experimentation. I've worked across traditional ML, NLP, computer vision, RAG, LLMs, agents, APIs, cloud deployment, and MLOps. I'm looking for an opportunity where I can use that end-to-end experience to build scalable AI products while also continuing to grow in areas like agentic AI, LLM evaluation, and production optimization.

### 1. AI-Powered Realtor Assistant

This project is basically a real estate application with an AI chatbot that helps people find properties without manually searching through hundreds of listings.

For example, instead of selecting many filters, a user can simply type: "I need a 2-bedroom apartment under \$2,500 near my office with parking." The chatbot understands the request, searches the available property information, and recommends suitable properties. I used Python and Django for the backend, React Native for the application, and the OpenAI API for the AI chatbot. I also created REST APIs to connect the application, chatbot, property data, and user authentication.

Simple interview explanation:

"I built an AI-powered real estate assistant where users can describe the type of property they are looking for in normal English. The chatbot understands their requirements and helps recommend relevant properties, which makes the property-search process much easier."

### 2. MediScan AI

MediScan AI is an image-based medicine identification system. The basic idea is that a user takes a picture of a medicine package, and the system tries to identify which medicine it is.

For example, suppose someone has a medicine box but doesn't clearly remember its name or purpose. They can take a picture using their phone. I used YOLOv8 and OpenCV to detect the medicine package and OCR to extract text such as the medicine name from the label. Once the medicine is identified, Generative AI can present useful information about it. For a real production healthcare application, dosage or safety guidance would need to come from verified medical sources rather than relying only on an LLM.

Simple interview explanation:

"I developed MediScan AI, where a user can take a picture of medicine and the system identifies it using computer vision and OCR. After identification, the application can retrieve and present information about that medicine in an easy-to-understand format."

### 3. AI PDF Study Assistant — RAG Project

This is probably the most directly relevant AI/LLM project on your resume. It is basically a ChatGPT-like application for your own PDF documents.

For example, imagine you upload a 100-page insurance policy PDF. Instead of reading the entire document, you can ask: "What is the maximum coverage for accidental damage?" The application searches the PDF for the most relevant sections and gives those sections to the LLM so it can generate an answer based on the document.

Technically, when a PDF is uploaded, I extract the text → split it into smaller chunks → create embeddings → store them in ChromaDB/FAISS → perform semantic similarity search when the user asks a question → retrieve relevant chunks → pass that context to the LLM → generate the final answer. This is what we call Retrieval-Augmented Generation, or RAG.

Simple interview explanation:

"I built a RAG-based PDF assistant where users upload documents and ask questions in natural language. Instead of sending the entire PDF to the LLM, I split the document into chunks, created embeddings, stored them in a vector database, and retrieved only the most relevant information for each question. For example, someone can upload a large insurance policy and ask about a specific coverage rule without reading the whole document."

Easy way to remember all three

Think of them like this:

Realtor Assistant: Text → AI → Property recommendation

MediScan AI: Image → Computer Vision/OCR → Medicine identification

PDF Assistant: PDF → RAG → Question answering

So these three projects demonstrate three different areas of AI: conversational AI, computer vision, and generative AI/RAG.